



Translated

From: e-maxx.ru

## Dynamic Programming on Broken Profile. Problem "Parquet"

Common problems solved using DP on broken profile include:

- finding number of ways to fully fill an area (e.g. chessboard/grid) with some figures (e.g. dominoes)
- finding a way to fill an area with minimum number of figures
- finding a partial fill with minimum number of unfilled space (or cells, in case of grid)
- finding a partial fill with the minimum number of figures, such that no more figures can be added

### Problem "Parquet"

**Problem description.** Given a grid of size  $N \times M$ . Find number of ways to fill the grid with figures of size  $2 \times 1$  (no cell should be left unfilled, and figures should not overlap each other).

Let the DP state be:  $dp[i, mask]$ , where  $i = 1, \dots, N$  and  $mask = 0, \dots, 2^M - 1$ .

$i$  represents number of rows in the current grid, and  $mask$  is the state of last row of current grid. If  $j$ -th bit of  $mask$  is 0 then the corresponding cell is filled, otherwise it is unfilled.

Clearly, the answer to the problem will be  $dp[N, 0]$ .

We will be building the DP state by iterating over each  $i = 1, \dots, N$  and each  $mask = 0, \dots, 2^M - 1$ , and for each  $mask$  we will be only transitioning forward, that is, we will be *adding* figures to the current grid.

### Implementation

```
int n, m;  
vector < vector<long long> > dp;
```